Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

in cooperation with

OFFIS

Design and implementation of a web-based and open-source hero administration for the role-playing game “Das Schwarze Auge” (Backend)

Bachelor thesis
Final presentation

Björn Nowak, 25.05.2023

1 Introduction
1.1 Motivation

Pen **&** **P**aper
Role-**P**laying **G**ame

“Das Schwarze Auge” (DSA) is germanys most popular P&P RPG

The problems:



Time-intensive and error-prone character creation and management for players

- A complex rulebook
  - Vast amount of rules
  - Different categories of rules for different game phases
  - Countless combinations of character feature forms
- Many rulebook extensions increasing complexity further

1 Introduction
1.1 Motivation

“Das Schwarze Auge” (DSA) is germanys most popular P&P RPG

The countermeasures:



Various tools supporting players keeping an overview and automating tasks

- E.g. character layouts, spread sheets and even programs
- Mainly (almost exclusively) created and published by the community

Current restriction <u>which this work tackles</u>:



Non of the popular programs supports the current edition reportingly due to required extensive work and lack of developing capacities

Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

a group of the department for computer science

in cooperation with

OFFIS

Outline

1 Introduction
- 1.1 Motivation
- 1.2 Objectives
- 1.3 Basics of “Das Schwarze Auge” (DSA)

2 Design
- 2.1 Used formalization method
- 2.2 Concept

3 Implementation
- 3.1 Character validation process & core components
- 3.2 Rulebooks logic programs (LPs)

4 Discussion and outlook

5 Conclusion

Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

a group of the department for computer science

in cooperation with

OFFIS

1 Introduction

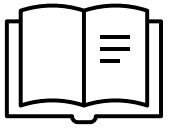
~~1.1 Motivation~~

1.2 Objectives

1.3 Basics of “Das Schwarze Auge” (DSA)

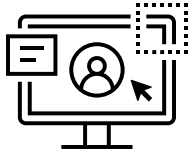
1 Introduction
1.2 Objectives

Creating a cookbook for formalizing rules of the current (5th) DSA edition, that…



– provides general conventions
– defines a character model
– outlines pre-conditions for processing

Creating a web-based and open-source software as a platform, which…

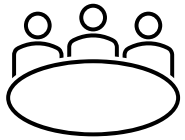


– implements above cookbook as a proof-of-concept
– enables easy migration to other editions
– supports multiple rulebook extensions
– is very maintainable

Overall focusing on base rules at the character creation/validation

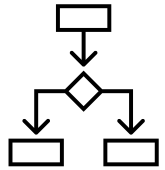


1 Introduction
1.3 Basics of “Das Schwarze Auge” (DSA)



Involved roles:

- One game master
- Characters, also called heros (remaining participants/players)

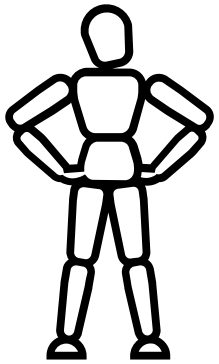


Game process:

- Participants create their character
- The game master leads the characters through a fictional world using a pre-defined rulebook (interactive story-telling)
- Characters may gain adventure points (AP) during a session and use this to advance themselves
- Eat, sleep and repeat

1 Introduction
1.3 Basics of “Das Schwarze Auge” (DSA)

Simplified specification of a character (by the base rulebook of the 5th edition [1])



one	[R]ace	of four
one	[C]ulture	of 25
one	[P]rofession	of 33
all	[CH]aracteristics	of eight
many	[S]kills	of 160
some	[SP]ecial talents	of 222
some	[A]dvantages	of 50
some	[D]isadvantages	of 52

- 59 talents
- 45 spells
- six rituals
- 16 cantrips
- 34 liturgies

e.g. **not** including combat techniques or calculated base values

$$\binom{R}{1} * \binom{C}{1} * \binom{P}{2} * \binom{CH}{6} * \binom{\frac{S}{4}+\frac{SP}{10}}{20} * \binom{\frac{A}{2}}{10} * \binom{\frac{D}{2}}{5} = \text{approx. } 3.16078 \text{ decillion } (10^{33}) \text{ combinations}$$

(equation includes assumptions)

Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

a group of the department for computer science

in cooperation with

OFFIS

2 Design

2.1 Used formalization method

2.2 Concept

2.2.1 Conventions

2.2.2 Input model

2.2.3 Preconditions

2.2.4 Cookbook for rule formalization

2.2.5 Result interpretation

2 Design
2.1 Used formalization method

Answer Set Programing (ASP)	
Principal	Finds solutions non-deterministic under constraints / rules
Language	Normal-form horn-clauses
Techniques	Stable Model Semantics (Answer Set), Grounder, Solver
Applications	Constraint-based modelling, Combinatorial optimization, planning problems and decision support
Commonness	Wider than Prolog and SAT *
Strengths	Precise and complete representation of knowledge, support of negation and uncertainty, complex program flow control
Weaknesses	Complex syntax and semantic hence steep learning curve

* Based on Google Scholar hits using the name and abbreviation

2 Design
2.1 Used formalization method

Used framework:



clingo
from Potassco (the Potsdam Answer Set Solving Collection)

Used programing language:



python

2 Design
2.1 Used formalization method

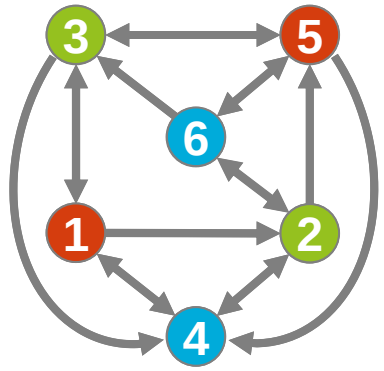
Typical usage / problem: (own representation following [2])
Colorize a directed graph such that no neighbor has the same color.

Problem instance:

```
# modelling the graph
node(1..6).
edge(1,2). edge(1,3). edge(1,4).
edge(2,4). edge(2,5). edge(2,6).
edge(3,1). edge(3,4). edge(3,5).
edge(4,1). edge(4,2).
edge(5,3). edge(5,4). edge(5,6).
edge(6,2). edge(6,3). edge(6,5).
# given three colors
color(r). color(g). color(b).
```

Problem solution:

```
# one possible answer set
assign(1,r). assign(2,g). assign(3,g).
assign(4,b). assign(5,r). assign(6,b).
```



Problem encoding:

```
# assign each node exactly one color
{ assign(N,C) : color(C) } = 1 :- node(N).
# but connected nodes shall not have the same color
:- edge(N,M), assign(N,C), assign(M,C).
```

2 Design
2.2 Concept / 2.2.1 Conventions

CV.1 It will be distinguished between character facts, world facts, world rules and result facts. Facts are clingo functions and world rules always derive result facts.



On world and result facts the first argument shall be the feature of which the used rule is from. These shall be called the causing feature.

[…]



Clingo functions with at least two arguments are to be read such that the function name describes the relation between these arguments.
E.g.: `relates_to(caused_feature,referred_feature).`

[…]

♡ have been proofed very well in the implementation phase [DC.7, DC.8].

2 Design
2.2 Concept / 2.2.2 Input model ∈ Problem instance

Provided by common.lp using the character domain model as context:

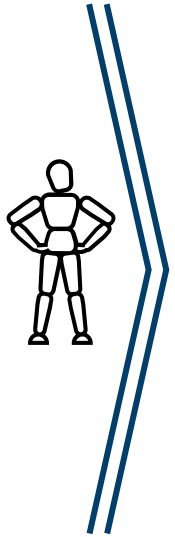
is always loaded

enables interaction between clingo/LP and the application

is api user input

external functions must be deterministic

```
1  #program character_facts.
2  __talent_wrapper(@talents).
3  __combat_technique_wrapper(@combat_techniques).
4  __advantage_wrapper(@advantages).
5  __disadvantage_wrapper(@disadvantages).
6  %% USES := free text reference, can be empty
7  experience_level(@experience_level).
8  race(@race).
9  culture(@culture).
10 profession(@profession).
11 talent(T,LVL) :-
       __talent_wrapper((T,LVL)).
12 combat_technique(CT,LVL) :-
       __combat_technique_wrapper((CT,LVL)).
13 advantage(A,USES,LVL) :-
       __advantage_wrapper ((A,USES,LVL)).
14 disadvantage(DA,USES,LVL) :-
       __disadvantage_wrapper((DA,USES,LVL)).
```



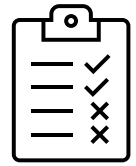
```
experience_level("Average").
race("Elfen").
culture("Auelfen").
profession("Söldner").
talent("Kraftakt",3).  talent("Zechen",3).

combat_technique("Armbrüste",10).

advantage("Begabung","Singen",1).

disadvantage("Wahrer Name","",1).
```

2 Design
2.2 Concept / 2.2.3 Preconditions



[PC.1] All rulebooks dependencies are present/used

Problem instance #2: (must be provided by specific rulebook LPs)

```
1  #program rulebook_usable.
2  rulebook("dsa5_aventurisches_götterwirken_2").
3  rulebook_depends("dsa5_aventurisches_götterwirken_2","dsa5").
```

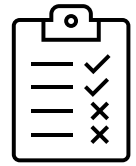
each rulebook is an own set of LPs

rulebook ≈ rulebook extension
base rulebook = has no dependencies

Problem encoding #2: (also provided by common.lp)

```
1  #program rulebook_usable.
2  rulebook_missing(RB,D) :- rulebook(RB), rulebook_depends(RB,D), not rulebook(D).
3  […]
```

2 Design
2.2 Concept / 2.2.3 Preconditions



[PC.2] All known feature forms must be declared first
[PC.3] Input model only uses known feature forms

Problem instance #2: (must be provided by specific rulebook LPs)

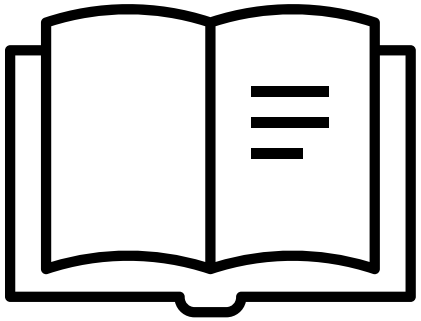
```
1  #program world_facts.
2  known_profession("Händler";"Söldner";"Zauberweber";"Wildnisläuferin").
3  known_advantage("Begabung",("Singen";"Musizieren"),(1..2)).
4  known_talent("Kraftakt";"Selbstbeherrschung";"Sinnesschärfe").
5  […]
```

Problem encoding #2: (also provided by common.lp)

```
1  #program validate_hero_step_50. %% pre check - values known
2  unknown(profession(P)) :- profession(P), not known_profession(P).
3  unknown(advantage(A,USES,LVL)) :- advantage(A,USES,LVL), not known_advantage(A,USES,LVL).
4  unknown(talent(T)) :- talent(T,_), not known_talent(T).
5  […]
```

2 Design
2.2 Concept / 2.2.4 Cookbook for rule formalization

Three groups of rules based on application-range and extent of compliance:



[GR] Rulebook-unspecific hard rules

– General cross rulebook rules

– Must be followed always

[HR] Rulebook-specific hard rules

– Must be followed always

[SR] Rulebook-specific soft rules

– Non-conformance possible on game master approval

2 Design
2.2 Concept / 2.2.4 Cookbook for rule formalization

[GR.3] An experience level applies restrictions like maximum levels and count

Problem encoding #3: (provided by common.lp)

```
1  %% Model: known_experience_level(<name>,<CL>,<SL>,<CTL>,<CP>,<SL>,<FS>).
2  %%  CL  := max. characteristic levels    %%  CP := max. characteristic points
3  %%  SL  := max. skill levels             %%  SL := max. spells and liturgies count
4  %%  CTL := max. combat technique levels  %%  FS := max. foreign spells count within above
5  known_experience_level("Average",13,10,10,98,10,1).
6  %%  e.g. find talents, which exceeds the maximum level
7  max_lvl_exceeded(talent(T,LVL),MAX) :-
       experience_level(EL),
       known_experience_level(EL,_,MAX,_,_,_,_),
       talent(T,LVL),
       LVL > MAX.
```

2 Design
2.2 Concept / 2.2.4 Cookbook for rule formalization

[HR.3] A feature form requires <u>one</u> form on a minimum level of another multi-form feature

Problem instance #4: (world fact provided by rulebooks)

```
1  requires(profession("Söldner"),talent("Kriegskunst",6)).
```

Problem encoding #4: (world rule provided by common.lp)

```
1  %% logical 'or' build with '#count': feature is missing or min. level is not reached
2  missing_level(profession(P),talent(T,MIN_LVL)) :-
       profession(P),
       requires(profession(P),talent(T,MIN_LVL)),
       1 = #count{ x: not talent(T,_) ; x: talent(T,LVL), LVL < MIN_LVL }.
```

```
1 = #count{   ;   }.   >>   1 = 0.   >>   #false
1 = #count{ x ;   }.   >>   1 = 1.   >>   #true
1 = #count{   ; x }.   >>   1 = 1.   >>   #true
1 = #count{ x ; x }.   >>   1 = 1.   >>   #true
```

2 Design
2.2 Concept / 2.2.4 Cookbook for rule formalization

[HR.4] A feature form requires a choice of forms of another multi-form feature on a min level

Problem instance #5: (world fact provided by rulebooks)

```
1  %% 'Söldner‘ requires two CTs of a list on at least level 10
2  requires(benötigt
         profession("Söldner"),
         any_of(2,combat_technique,("Hiebwaffen","Schwerter","Stangenwaffen"),10)
   ).
```

Problem encoding #5: (world rule provided by common.lp)

```
1  missing_level(profession(P),combat_technique(any_of(CHOICES,CTs),MIN_LVL))  :-
         profession(P),
         requires(profession(P),any_of(CHOICES,combat_technique,CTs,MIN_LVL)),
         CHOICES > @count_by("combat_techniques",CTs,MIN_LVL).
```

2 Design
2.2 Concept / 2.2.4 Cookbook for rule formalization

[HR.4] A feature form requires a choice of forms of another multi-form feature on a min level

```
1  from clingo import Symbol, Number
2  ## method of context class (input model) having the character as class field (self.hero)
3  def count_by(self, feature: Symbol, options: Symbol, min_lvl: Symbol) -> Symbol:
4    """
5    :return: count of 'options' (feature forms) of 'feature' having minimum level
6    """
7    ## dynamically get class field (with 'getattr') instead of manual mapping with a switch-case
8    ##   requires 'feature.string' to be exactly the field name of the actual hero model
9    values_lvl: dict[str, int] = {fv.name: fv.level for fv in getattr(self.hero, feature.string)}
10   ## count all features having the minimum level
11   passed = sum(1 for opt in options.arguments if min_lvl.number <= values_lvl.get(opt.string, 0))
12   return Number(passed)
```

2 Design
2.2 Concept / 2.2.4 Cookbook for rule formalization

[SR.2] A feature form declares a form of another feature as typical

Problem instance #6: (world fact provided by rulebooks)

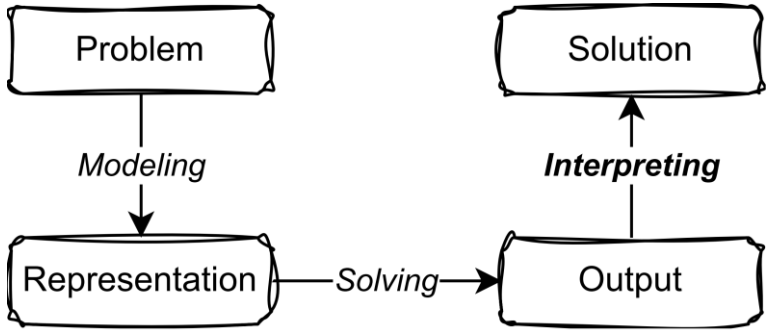
```
1  has_typical(race("Elfen"),advantage("Dunkelsicht","",2)).
```

Problem encoding #6: (world rule provided by common.lp)

```
1  missing_typical(race(R),advantage(A,USES,LVL))  :-
       race(R),
       has_typical(race(R),advantage(A,USES,LVL)),
       not advantage(A,USES,LVL).
```

2 Design
2.2 Concept / 2.2.5 Result interpretation

General process: (own representation following [2])



Unusual for ASP:

- Not searching for all possible answer sets
- All problems (LPs) are satisfiable, enabling to…
- …always produce “fixed” result facts
  e.g. problem encoding #6:

```
1  missing_typical(race(R),advantage(A,USES,LVL))  :-
       race(R),
       has_typical(race(R),advantage(A,USES,LVL)),
       not advantage(A,USES,LVL).
```

Carl von Ossietzky
Universität
Oldenburg

Digitalisierte
Energiesysteme

a group of the department
for computer science

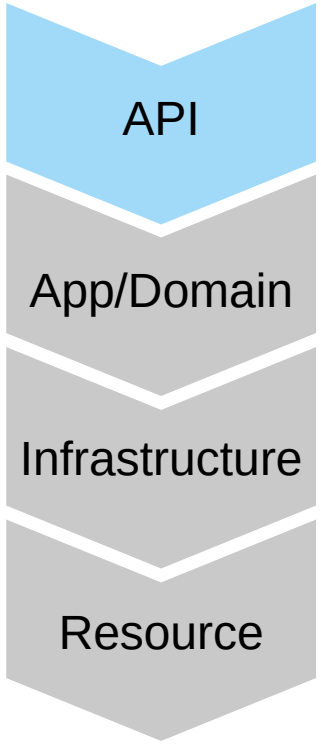
in cooperation with

OFFIS

3 Implementation

~~3.0 Architecture and design decisions~~
3.1 Character validation process & core components
3.2 Rulebooks logic programs (LPs)
~~3.3 Architecture and design decisions~~
~~3.4 Encounters difficulties~~

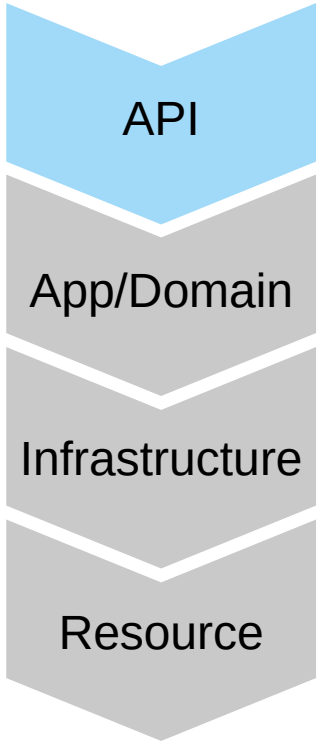
3 Implementation
3.1 Character validation process & core components



Example request (json): `<host>/api/hero/validate?rulebooks=dsa5`

```
{
  "name": "UncleBob",
  "experience_level": "Legendary",
  "race": "Elfen",
  "culture": "Auelfen",
  "profession": "Söldner",
  "talents": {
    "Körperbeherrschung": 2,
    "Kraftakt": 3,
    "Selbstbeherrschung": 4,
    "Zechen": 5,
    "Menschenkenntnis": 3,
    "Überreden": 3,
    "Orientierung": 4,
    "Wildnisleben": 3,
    "Götter & Kulte": 3,
    "Kriegskunst": 6,
    "Sagen & Legenden": 5,
    "Handel": 3,
    "Heilkunde Wunden": 4
  },
  "combat_techniques": {
    "Armbrüste": 10,
    "Raufen": 10,
    "Stangenwaffen": 9,
    "Zweihandschwerter": 10
  },
  "advantages": [
    ["Begabung", "Singen", 1],
    ["Begabung", "Musizieren", 1],
    ["Beidhändig", "", 1],
    ["Dunkelsicht", "", 2]
  ],
  "disadvantages": [
    ["Körpergebundene Kraft", "", 1],
    ["Lästige Mindergeister", "", 1],
    ["Wahrer Name", "", 1]
  ]
}
```

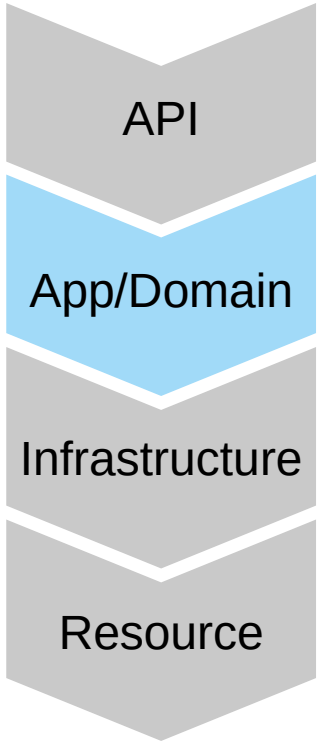
3 Implementation
3.1 Character validation process & core components



Example response (json): `<host>/api/hero/validate?rulebooks=dsa5`

```
{
  "valid": false,
  "errors": [
    {
      "type": "missing_level",
      "message": "Heros 'profession' is missing minimum level '3'
                  for 'talent' of 'Körperbeherrschung'.",
      "parameter": {
        "caused_feature": "profession",
        "caused_feature_value": "Söldner",
        "referred_feature": "talent",
        "referred_feature_value": "Körperbeherrschung",
        "min_level": 3
      }
    }
  ],
  "warnings": []
}
```

3 Implementation
3.1 Character validation process & core components

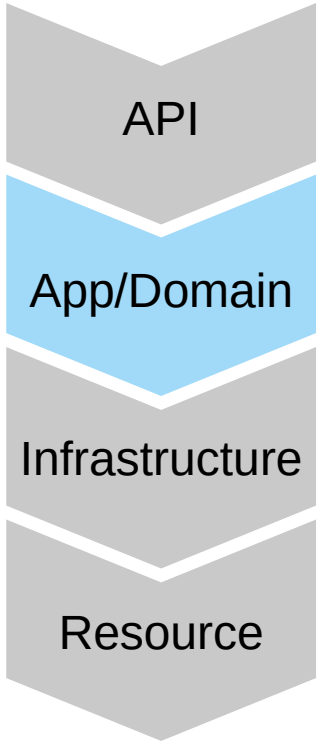


Engine (core component):

- Defines which LP files are to be loaded
- Defines which LP sub-programs are to be executed
- Fails whenever given rulebook set is not usable
- Executes character validation step-wise (each one is a sub-program)
  - Default: pre-conditions > feature usable > feature requirements >
  - Detects addition steps within rulebooks LPs

for each feature;
fixed order
race > culture > …

3 Implementation
3.1 Character validation process & core components

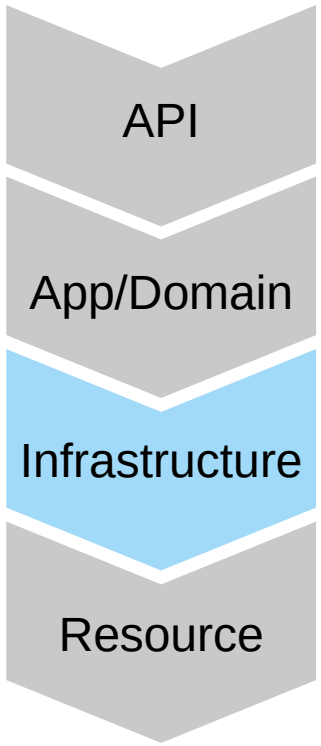


Engine

Collector & **HeroValidationInterpreter** (core components):

- Retrieves result facts from clingo answer set
- Interprets result facts (and their arguments)
- Convert result facts into domain models:
  for hard rules: for soft rules:
  - HeroValidationError and HeroValidationWarning
  - Stores all result fact arguments in a map of constant parameters
    enables API users (like a frontend) further processing

3 Implementation
3.1 Character validation process & core components

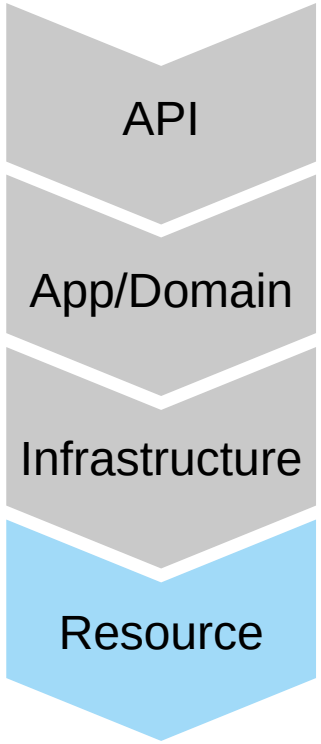


Engine

ClingoExecutor (core component):

- Interface between application and ASP framework (loose coupling)
- Loads, ground and execute given LPs and sub-programs
  using the clingo api
- Fails whenever run was not satisfiable
- Accepts an optional context
  - Uses a convenient wrapper for the character model
    convert python datatypes to clingo symbols
- Returns complete model (all symbols; could be reduced to result facts)

3 Implementation
3.1 Character validation process & core components



ClingoExecutor

Rulebook LPs:

– Each rulebook is an own set of LPs

– Contained in separate folders

– Required to have an “_entrypoint.lp” top-level file

the single file of a rulebook directly used by the engine;
an LP can extend itself by clingo directive “#include”

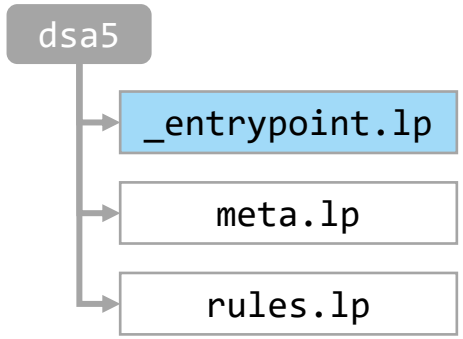
– Required to have specific sub-programs (used by the Engine)

don’t forget the common.lp

and remember:
rulebook ≈ rulebook (extension)

3 Implementation
3.2 Rulebooks logic programs (LPs)

File structure:

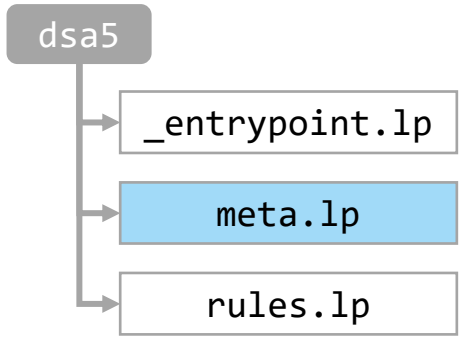


File content:

```
1  #include "meta.lp".
2  #include "rules.lp".
```

3 Implementation
3.2 Rulebooks logic programs (LPs)

File structure:

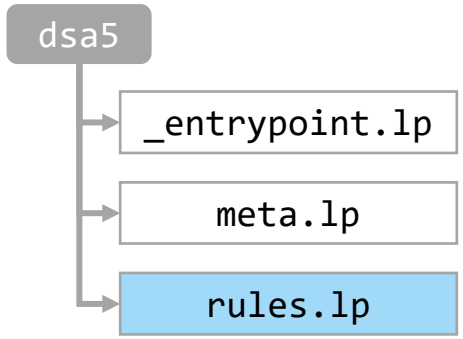


File content:

```
1  #program rulebook_usable.
2  program("rulebook_usable").
3  %% make presence of this rulebook available for others
4  rulebook("dsa5").
5  %% this rulebook depends on nothing
6
7  #program world_facts.
8  program("world_facts").
9
10 known_experience_level("Unerfahren",       12, 10,  8, 95,  8, 0 ).
11 known_experience_level("Durchschnittlich", 13, 10, 10, 98, 10, 1 ).
12 […]
13 known_race("Elfen"; […]).
14 known_culture("Auelfen"; […]).
15 known_advantage("Begabung",("Singen";"Musizieren"),1).
16 known_disadvantage(("Prinzipientreue";"Verpflichtungen"),"",(1..3)).
17 known_talent("Zechen";"Kraftakt";"Überreden"; […]).
18 known_combat_technique("Armbrüste";"Raufen";"Hiebwaffen"; […]).
19 […]
20 %% pre-conditions
21 max_count(race,1).
22 max_count(culture,1).
23 max_count(profession,1).
```

3 Implementation
3.2 Rulebooks logic programs (LPs)

File structure:

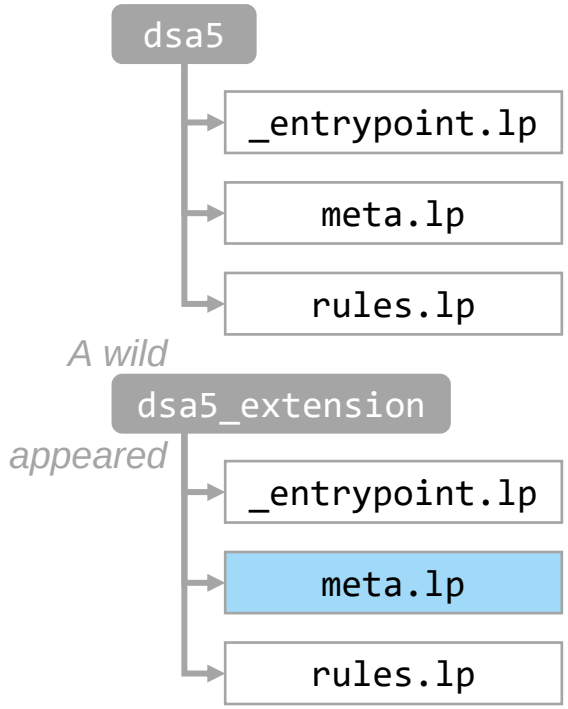


File content:

```
#program world_facts.
program("world_facts").
%% race Elfen
requires(race("Elfen"),advantage("Zauberer","",1;"Zweistimmiger Gesang","",1)).
has_usual(race("Elfen"),culture("Auelfen";"Firnelfen";"Waldelfen")).
has_typical(race("Elfen"),advantage("Begabung",("Singen";"Musizieren"),1; […]).
has_atypical(race("Elfen"),disadvantage("Blutrausch","",1; […]).
%% race Halbelfen
[…]
%% culture to race
requires(culture("Auelfen";"Firnelfen"; […]),race("Elfen")).
[…]
%% culture Auelfen
has_usual(culture("Auelfen"),profession("Wildnisläufer";"Zauberweber")).
[…]
%% profession Söldner
requires(profession("Söldner"),talent("Körperbeherrschung",3; […]).
requires(profession("Söldner"),combat_technique("Armbrüste",10;"Raufen",10)).
requires(profession("Söldner"),any_of(1,combat_technique,("Hiebwaffen", […]),10)).
[…]
```

3 Implementation
3.2 Rulebooks logic programs (LPs)

File structure:



File content:

```
1  #program rulebook_usable.
2  program("rulebook_usable").
3  %% make presence of this rulebook available for others
4  rulebook("dsa5_extension").
5  %% this rulebook depends on base ruelbook dsa5
6  rulebook_depends("dsa5_extension","dsa5").
7
8  #program world_facts.
9  program("world_facts").
10 known_culture("Fjarninger";"Mhanadistani").
11 known_profession("Skuldrun";"Säbeltänzerin").
12 […]
```

Carl von Ossietzky Universität Oldenburg

Digitalisierte Energiesysteme

a group of the department for computer science

in cooperation with

OFFIS

4 Discussion and outlook

4 Discussion and outlook
A pre-discussion

[DC.2] Features could be grouped to reduce repetition in rules

(was yet not required due to few groupable features)

```
1  %% Example of a repetition
2  missing_min_lvl(talent(T,LVL),MIN) :- talent(T,LVL), MIN = 0, LVL < MIN .
3  missing_min_lvl(combat_technique(CT,LVL),MIN) :- combat_technique(CT,LVL), MIN = 0, LVL < MIN .
4  %% Group option A: additionally to character facts
5  skill(talent,NAME,LVL) :- talent(NAME,LVL).
6  skill(combat_technique,NAME,LVL) :- combat_technique(NAME,LVL).
7  %% Group option B: replace character facts (not recommended)
8  skill(talent,NAME,LVL) :- __talent_wrapper((NAME,LVL)).
9  skill(combat_technique,NAME,LVL) :- __combat_technique_wrapper((NAME,LVL)).
10 %% Result: single rule possible
11 missing_min_lvl(FEATURE,NAME,LVL,MIN) :- skill(FEATURE,NAME,LVL), MIN = 0, LVL < MIN.
```

4 Discussion and outlook
A pre-discussion

[DC.4] The "`@count_by`" could not be formalized so that it is easy to understand and maintain

(no real issue: external function only implements simple logic)

```
1  %%%% 'prof := profession' und 'combatT := combat_technique'
2  %% Choice list with three elements having the same key
3  requires(prof("Söldner"),any_of("uniqueID",1,combatT("Hiebwaffen"; […]),10)).
4  %% Approach using "head  aggregation": creates three result facts with same key
       in order to compose a choice list on result interpretation
5  CHOICES > #count{
       KEY,CT
           : missing_level(prof(P),any_of(KEY,CHOICES,combatT,CT),MIN_LVL))
           : requires(prof(P),any_of(KEY,CHOICES,combatT(CT),MIN_LVL)),
               combatT(CT,LVL),
               LVL >= MIN_LVL
   } :-
       prof(P),
       requires(prof(P),any_of(KEY,CHOICES,combatT(_),MIN_LVL)).
```

approach implementation is not a valid LP and hence did not work (yet)

4 Discussion and outlook
An outlook

[OL.1] There shall be a technical validation (rule) that let rulebook LPs only use known feature forms (in respect of their dependencies)

[OL.2] Character validation errors and warnings shall also state (if possible) what would be right (like which feature forms are available under selected “higher” features)

[OL.3] Character validation warnings shall be ignorable as game master approved

[OL.4] Character management: leveling up using APs

- Keep track of received and spent APs
- Successive validation of each change (“multi-shot solving” ?)

and more like user registration and storing characters in a database

5 Conclusion

5 Conclusion

Formalizing the DSA rulebook is possible

…and easy to implement and understand

…but future will show if this holds when

…all rules and more rulebooks are implemented

…and especially character management.

Besides some (intentional) easy-to-fix shortcomings the software

…is maintainable and extensible

…and could even support other DSA editions

hence it is a good starting point for the community.

May soon available for contributions at:
https://github.com/bjoern-nowak/dsa-heldenbogen

6 Literature

[1] Ulisses Spiele GmbH: „DSA Regel Wiki“. Aus Unterseiten. 2023. url: https://www.ulisses-regelwiki.de/start.html

[2] Martin Gebser and others: “Answer Set Solving in Practice”. December 2012. doi:10.2200/S00457ED1V01Y201211AIM019: